

merrypopins: A Python package for nanoindentation data science

Cahit Acar¹, Anna Marcelissen¹, Hugo van Schrojenstein Lantman¹, and John M. Aiken^{1,2,3¶}

¹ Utrecht University, The Netherlands ² Expert Analytics, Norway ³ University of Oslo, Njord Centre, Norway ¶ Corresponding author

DOI: [10.21105/joss.09933](https://doi.org/10.21105/joss.09933)

Software

- [Review](#) ↗
- [Repository](#) ↗
- [Archive](#) ↗

Editor: Fei Tao ↗ 

Reviewers:

- [@KennethKwabenaKenney](#)
- [@Manishms18](#)

Submitted: 22 July 2025

Published: 14 August 2026

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

merrypopins is a Python library to streamline the workflow of nanoindentation experiment data processing, automated pop-in detection, and statistical analysis of collections of pop-in events. Nanoindentation is a technique for experimental deformation of materials with the aim of characterizing material behaviour and quantifying mechanical properties from load-displacement data ([Oliver & Pharr, 1992, 2004](#)). Experiments performed with a spherical tip can also be used to construct stress-strain curves and by extension the determination of the yield point defining the transition from elastic to plastic deformation ([Kalidindi & Pathak, 2008](#); [Pathak & Kalidindi, 2015](#)). Understanding the start of plasticity in materials at the microscale is crucial for various applications, including engineered materials and earthquake mechanics. A common feature during the loading part of nanoindentation experiments is the sudden increase of indentation depth at constant force, called “pop-in” events. Manually recognizing these characteristics is labor-intensive and subjective, emphasizing the importance of automated, reproducible detection approaches.

Statement of need

Detecting pop-ins is difficult because they appear in subtle, intermittent, and different ways within indentation curves. Historically, professional analysts have recognized pop-in occurrences manually. The researcher simply looks at either depth vs. time or stress-strain curves looking for sharp, localized changes. This approach suffers from subjectivity, labor intensity, and potential inconsistencies among multiple observers and big datasets. Modern nano-indentation machines can perform up to 12 indentations per second ([Bruker, n.d.](#)), so the volume of data has outgrown what manual inspection can keep up with.

Pop-ins are linked to dislocation in crystalline materials and are considered small-scale analogues of earthquakes ([Ispánovity et al., 2022](#); [Sato et al., 2020](#)). Like real earthquakes, they follow statistical patterns, such as power-law distributions in size and time between events. Generally, the first pop-in during an indentation experiment coincides with the start of plasticity. The size of the indenter tip and the degree of pre-existing plastic deformation have a significant impact on the stress at which the first pop-in occurs ([Morris et al., 2011](#); [Shim et al., 2008](#)) and thus the yield hardness of the material. This effect is the result of a delayed plastic yielding when the volume stressed by the indenter tip does not contain any pre-existing dislocations for the initiation of plasticity. A smaller volume is more likely to be free of dislocations, especially when the material has a lower dislocation density, so the material will behave elastically up to higher load and stress. In contrast, larger tips sample a bigger volume, increasing the chance of hitting existing dislocations and causing the first pop-in at lower stresses. This size effect must be overcome to obtain yield hardness values applicable across scales or to other systems.

Recovering these statistics requires every event along a curve, not just the first one, which is precisely what manual picking scales worst at.

Primary users of merrypopins are students, researchers, and academics in the fields of material science, geology, nano-mechanics, and earthquake science. High-resolution indentation experiments are increasingly used to investigate plastic and fracture processes at the microscale. Despite the growing number of studies targeting pop-in occurrences in load-depth curves, almost all previous research relies on manual inspection or private scripts with undisclosed methods for the detection and quantification of pop-ins, creating a lack of easily accessible, reproducible event detection software. There is an urgent need for adaptable, open-source solutions that can be used “out of the box” by non-programmers and provide extensibility for power users as nanoindentation tools grow, spanning both traditional materials laboratories and emerging geophysical applications. To advance the next generation of automated pop-in analysis, researchers can submit new detection techniques, parameter settings, or visualization modules through our public merrypopins GitHub repository. We, therefore, welcome feature requests, bug reports, and community-contributed enhancements.

State of the field

Software for instrumented indentation falls into three groups.

Vendor software shipped with commercial indenters, such as the suite supplied with the Bruker Hysitron platform ([Bruker, n.d.](#)), acquires curves and computes hardness and modulus. It is closed source, tied to one instrument, and exposes no pop-in event extraction, so analyses built on it cannot be reproduced or extended outside the vendor’s environment.

Open-source indentation packages target property extraction rather than event detection. micromechanics ([Brinckmann & Micromechanics Team, 2026](#)) is a Python library that reads a wide range of vendor formats and evaluates hardness and Young’s modulus by the Oliver-Pharr method ([Oliver & Pharr, 1992, 2004](#)), including frame stiffness and area function calibration. It has no pop-in detection layer. It solves an adjacent problem well, and merrypopins deliberately does not duplicate it.

The closest prior work is the PopIn toolbox ([Mercier, 2023](#)), a MATLAB package that fits the Hertz model to load-displacement curves and plots Weibull or time and temperature dependent cumulative distributions of the first or second pop-in. Two things make it a poor foundation for our target audience. It requires a commercial MATLAB licence, which students and geoscience groups often do not have, and it is scoped to the first and second pop-in of a curve rather than to every event along it. Power-law size and waiting-time statistics need the full catalogue of events.

Machine learning has been applied to this problem before. Kossman & Bigerelle ([2021](#)) trained a convolutional neural network that classifies whole load-displacement curves as containing a pop-in or not, reaching roughly 93% accuracy. That answers a different question, whether a curve contains a pop-in rather than where each pop-in sits, it needs labelled training curves, and the code was not released, so it cannot be reused or reproduced.

merrypopins therefore fills a gap rather than reimplementing existing tools. Its contribution is fourfold: per-point localisation of every pop-in along a curve instead of per-curve classification; four complementary detectors behind one interface, with an explicit cross-method agreement score; a statistical suite that turns located events into the stress-strain, precursor, and temporal quantities that pop-in studies report; and a no-code Streamlit app for users who do not program. All of it is MIT licensed Python installable from PyPI.

Software design

merrypopins is five modules that pass pandas DataFrames along a pipeline: `load_datasets` parses vendor .txt curves and .tdm/.tdx metadata, `preprocess` trims setup artefacts and zeroes the depth axis at the detected contact point, `locate` detects pop-ins, `statistics` computes stress-strain curves, yield points, and precursor and temporal statistics, and `make_dataset` runs the first three end to end and writes annotated tables with overlay plots. Because every stage consumes and returns a DataFrame, users can stop anywhere, inspect the intermediate result, or substitute their own step. This was a deliberate trade-off. A single opaque `analyse()` call would be simpler to document, but researchers need to see and correct what the contact-point detector did before they will trust anything downstream of it.

Choosing and combining detection methods. Using a variety of detection techniques ensures that merrypopins can detect pop-in events in many material systems and experimental circumstances. Savitzky-Golay local polynomial smoothing maintains prominent curve characteristics while reducing high-frequency noise (Savitzky & Golay, 1964), and Fourier spectral differentiation identifies abrupt discontinuities with minimal parameterization (Cooley et al., 1969). Both are computationally efficient, highly interpretable, and require only a few user-tunable parameters, which makes them a good first pass over a large batch. Two unsupervised learning methods add data-driven adaptation. Isolation Forest detects anomalies in multidimensional feature spaces without labeled instances (Liu et al., 2008), which helps when pop-in magnitudes or frequencies are unknown beforehand, and a convolutional autoencoder learns feature representations directly from the curve, capturing subtle nonlinear patterns that a fixed derivative threshold overlooks (Malhotra et al., 2016), at a higher computational cost. Rather than asking the user to pick one, `default_locate` runs the enabled methods and reports all of them: each writes its own boolean column, `popin` is their union, `popin_score` counts how many methods fired at a point, and `popin_confident` requires agreement from at least two. Users who need recall take `popin`; users for whom a false positive is more costly than a missed event take `popin_confident`.

No pretrained models. Neither learning method ships with weights. The Isolation Forest and the autoencoder are both fitted, unsupervised, on the curve being analysed at call time, using local stiffness difference and curvature as features. No labelled training data is required, and a result depends only on the curve supplied and the parameters chosen. All four methods are restricted to the loading portion of the curve, up to maximum load, because unloading artefacts are irrelevant to pop-in analysis. TensorFlow, needed only by the autoencoder, is an optional install extra so that the other three methods carry no heavyweight dependency.

Validating detections. Because pop-in ground truth is itself a matter of expert judgement, merrypopins is built so detections can be checked rather than merely trusted. `popin_score` makes cross-method agreement visible, and events that only one detector sees can be flagged for inspection. `make_dataset` writes an overlay plot per curve with each method's detections marked in a distinct colour, so a whole batch can be reviewed visually. The repository ships 36 real indentation curves with their metadata and rendered overlays, five tutorial notebooks that reproduce the single-file and multi-file workflows end to end, and a test suite covering the five modules, including synthetic curves with pop-ins of known position and size.

Development approach. The library was developed using a tutorial-driven software development framework (Aiken, 2020; Aiken et al., 2025; Woods et al., 2022). Instead of starting with predetermined architectural specs, this approach converts the scientist's process into a live, executable lesson, often a Jupyter notebook. Developers and researchers worked iteratively, with academics creating function stubs in a scientific narrative framework and developers implementing these functions based on real-world usage cases. This strategy ensures that scientific usability drives software design.

Code Availability

The merrypopins package can be installed via:

```
pip install merrypopins          # three detectors, lightweight
pip install 'merrypopins[cnn]'   # adds the autoencoder, pulls in TensorFlow
```

Alternatively, the package can be found on GitHub (<https://github.com/SerpRateAI/merrypopins>), where the datasets, tutorial notebooks, and test suite described above are also available.

Contributions can be made by forking the repository and making a pull request.

The Streamlit app is accessible via the Streamlit website (<https://merrypopins.streamlit.app/>).

AI usage disclosure

The merrypopins library, its test suite, its documentation, and the original version of this manuscript were written by the authors without generative AI assistance.

During the revision of this manuscript in response to peer review, an AI coding assistant was used for three things: drafting prose for the “State of the field” and “Software design” sections from sources the authors selected and checked, adding type annotations to existing function signatures, and refactoring TensorFlow into an optional install extra. Every resulting change was reviewed, tested, and accepted by the authors. All scientific claims, design decisions, comparisons with other software, and the interpretation of results are the authors’ own, and the authors take full responsibility for the content of both the software and this paper.

Acknowledgements

This project has received funding from the Norwegian Research Council (SerpRateAI, grant no. 334395) and is supported by EPOS-eNLarge funded by the Dutch Research Council (NWO) Roadmap for large-scale research infrastructure. We would like to thank Alissa Kotowski for fruitful conversations.

References

- Aiken, J. M. (2020). Science coding vs software development. In *substack*. substack. <https://mnky9800n.substack.com/p/science-coding-vs-software-development>
- Aiken, J. M., Jones, G., Yin, X., Abele, A. K., Woods, C., Westaway, R. M., & Bamber, J. L. (2025). 4DModeller: A spatio-temporal modelling package. *Journal of Open Source Software*, 10(106), 7047. <https://doi.org/10.21105/joss.07047>
- Brinckmann, S., & Micromechanics Team. (2026). *Micromechanics: Investigation of nanoindentation and SEM data*. GitHub. <https://micromechanics.github.io/micromechanics/>
- Bruker. (n.d.). *Hysitron TI 990 TriboIndenter brochure*. Bruker. <https://www.bruker.com/en/meta/forms/bns-form-pages/brochures/ni/hysitron-ti-990.html>
- Cooley, J. W., Lewis, P. A. W., & Welch, P. D. (1969). The fast fourier transform and its applications. *IEEE Transactions on Education*, 12(1), 27–34. <https://doi.org/10.1109/TE.1969.4320436>
- Ispánovity, P. D., Ugi, D., Péterffy, G., Knappek, M., Kalácska, S., Tüzes, D., Dankházi, Z., Máthis, K., Chmelík, F., & Groma, I. (2022). Dislocation avalanches are like earthquakes on

- the micron scale. *Nature Communications*, 13(1), 1975. <https://doi.org/10.1038/s41467-022-29044-7>
- Kalidindi, S. R., & Pathak, S. (2008). Determination of the effective zero-point and the extraction of spherical nanoindentation stress-strain curves. *Acta Materialia*, 56(14), 3523–3532. <https://doi.org/10.1016/j.actamat.2008.03.036>
- Kossman, S., & Bigerelle, M. (2021). Pop-in identification in nanoindentation curves with deep learning algorithms. *Materials*, 14(22), 7027. <https://doi.org/10.3390/ma14227027>
- Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation forest. *2008 Eighth IEEE International Conference on Data Mining*, 413–422. <https://doi.org/10.1109/ICDM.2008.17>
- Malhotra, P., Ramakrishnan, A., Anand, G., Vig, L., Agarwal, P., & Shroff, G. (2016). *LSTM-based encoder-decoder for multi-sensor anomaly detection*. <https://doi.org/10.48550/arXiv.1607.00148>
- Mercier, D. (2023). *PopIn: "Pop-in" analysis from indentation tests*. GitHub. <https://github.com/DavidMercier/PopIn>
- Morris, J. R., Bei, H., Pharr, G. M., & George, E. P. (2011). Size effects and stochastic behavior of nanoindentation pop in. *Physical Review Letters*, 106(1), 165502. <https://doi.org/10.1103/PhysRevLett.106.165502>
- Oliver, W. C., & Pharr, G. M. (1992). An improved technique for determining hardness and elastic modulus using load and displacement sensing indentation experiments. *Journal of Materials Research*, 7(6), 1564–1583. <https://doi.org/10.1557/JMR.1992.1564>
- Oliver, W. C., & Pharr, G. M. (2004). Measurement of hardness and elastic modulus by instrumented indentation: Advances in understanding and refinements to methodology. *Journal of Materials Research*, 19(1), 3–20. <https://doi.org/10.1557/jmr.2004.19.1.3>
- Pathak, S., & Kalidindi, S. R. (2015). Spherical nanoindentation stress-strain curves. *Materials Science and Engineering R*, 91(1), 1–36. <https://doi.org/10.1016/j.mser.2015.02.001>
- Sato, Y., Shinzato, S., Ohmura, T., Hatano, T., & Ogata, S. (2020). Unique universal scaling in nanoindentation pop-ins. *Nature Communications*, 11(1), 4177. <https://doi.org/10.1038/s41467-020-17918-7>
- Savitzky, Abraham., & Golay, M. J. E. (1964). Smoothing and differentiation of data by simplified least squares procedures. *Analytical Chemistry*, 36(8), 1627–1639. <https://doi.org/10.1021/ac60214a047>
- Shim, S., Bei, H., George, E. P., & Pharr, G. M. (2008). A different type of indentation size effect. *Scripta Materialia*, 59(1), 1095–1098. <https://doi.org/10.1016/j.scriptamat.2008.07.026>
- Woods, C., Hedges, L., Edsall, C., Brooks-Pollock, E., Parton-Fenton, C., McKinley, T. J., Keeling, M. J., & Danon, L. (2022). MetaWards: A flexible metapopulation framework for modelling disease spread. *Journal of Open Source Software*, 7(70), 3914. <https://doi.org/10.21105/joss.03914>